riko Cookbook

Index

User input

Some workflows require user input (via the `pipeinput` pipe). By default, `pipeinput` prompts the user via the console, but in some situations this may not be appropriate, e.g., testing or integrating with a website. In such cases, the input values can instead be read from the workflow's `inputs` kwargs (a set of values passed into every pipe).

```
>>> from riko.modules.input import pipe
>>> conf = {'prompt': 'How old are you?', 'type': 'int'}
>>> next(pipe(conf=conf, inputs={'content': '30'}))
{'content': 30}
```

Fetching data and feeds

`riko` can read both local and remote filepaths via `source` pipes. All `source` pipes return an equivalent `feed` iterator of dictionaries, aka `items`.

```
>>> from itertools import chain
>>> from riko import get_path
>>> from riko.modules import (
...     fetch, fetchdata, fetchsitefeed, feedautodiscovery)
>>>
>>> ### Fetch a url ###
>>> stream = fetchpage.pipe(conf={'url': 'https://news.ycombinator.com'})
>>>
>>> ### Fetch a filepath ###
>>> #
>>> # Note: `get_path` just looks up a file in the `data` directory
>>> stream = fetchdata.pipe(conf={'url': get_path('quote.json')})
>>>
>>> ### View the fetched data ###
>>> item = next(stream)
>>> item['list']['resources'][0]['resource']['fields']['symbol']
'KRW=X'

>>> ### Fetch an rss feed ###
>>> stream = fetch.pipe(conf={'url': 'https://news.ycombinator.com/rss'})
>>>
>>> ### Fetch the first rss feed found ###
>>> stream = fetchsitefeed.pipe(conf={'url': 'http://www.bbc.com/news'})
>>>
>>> ### Find all rss links and fetch the feeds ###
>>> url = 'http://edition.cnn.com/services/rss'
>>> entries = feedautodiscovery.pipe(conf={'url': url})
>>> urls = (e['link'] for e in entries)
>>> stream = chain.from_iterable(fetch(conf={'url': url}) for url in urls)
>>>
```

```
>>> ### Alternatively, create a SyncCollection ###
>>> #
>>> # `SyncCollection` is a url fetching convenience class with support for
>>> # parallel processing
>>> from riko.collections import SyncCollection
>>>
>>> sources = [{'url': url} for url in urls]
>>> stream = SyncCollection(sources).fetch()
>>>
>>> ### View the fetched rss feed(s) ###
>>> #
>>> # Note: regardless of how you fetch an rss feed, it will have the same
>>> # structure
>>> item = next(stream)
>>> sorted(item.keys())
[
    'author', 'author.name', 'author.uri', 'comments', 'content',
    'dc:creator', 'id', 'link', 'pubDate', 'summary', 'title',
    'updated', 'updated_parsed', 'y:id', 'y:published', 'y:title']
>>> item['title'], item['author'], item['link']
(
    u'Using NFC tags in the car', u'Liam Green-Hughes',
    u'http://www.greenhughes.com/content/using-nfc-tags-car')
```

Alternate `conf` value entry

Some workflows have `conf` values that are wired from other pipes

```
>>> from riko import get_path
>>> from riko.modules.fetch import pipe
>>>
>>> conf = {'url': {'subkey': 'url'}}
>>> result = pipe({'url': get_path('feed.xml')}, conf=conf)
>>> set(next(result).keys()) == {
...     'updated', 'updated_parsed', 'pubDate', 'author', 'y:published',
...     'title', 'comments', 'summary', 'content', 'link', 'y:title',
...     'dc:creator', 'author.uri', 'author.name', 'id', 'y:id'}
True
```

Alternate workflow creation

In addition to class based workflows `riko` supports a pure functional style [1].

```
>>> import itertools as it
>>> from riko import get_path
>>> from riko.modules import fetchpage, strreplace, tokenizer, count
>>>
>>> ### Set the pipe configurations ###
>>> #
>>> # Notes:
>>> #   - `get_path` just looks up files in the `data` directory to simplify
>>> #     testing
>>> #   - the `detag` option will strip all html tags from the result
>>> url = get_path('users.jyu.fi.html')
>>> fetch_conf = {'url': url, 'start': '<body>', 'end': '</body>', 'detag': True}
```

```
>>> replace_conf = {'rule': {'find': '\n', 'replace': ' '}}
>>>
>>> ### Create a workflow ###
>>> #
>>> # The following workflow will:
>>> #   1. fetch the url and return the content between the body tags
>>> #   2. replace newlines with spaces
>>> #   3. tokenize (split) the content by spaces, i.e., yield words
>>> #   4. count the words
>>> #
>>> # Note: because `fetchpage` and `strreplace` each return an iterator of
>>> # just one item, we can safely call `next` without fear of loosing data
>>> page = next(fetchpage.pipe(conf=fetch_conf))
>>> replaced = next(strreplace.pipe(page, conf=replace_conf, assign='content'))
>>> words = tokenizer.pipe(replaced, conf={'delimiter': ' '}, emit=True)
>>> counts = count.pipe(words)
>>> next(counts)
{'count': 70}

>>> from itertools import chain
>>> from riko import get_path
>>> from riko.modules import fetch, filter, subelement, regex, sort
>>>
>>> ### Set the pipe configurations ###
>>> #
>>> # Note: `get_path` just looks up files in the `data` directory to
>>> # simplify testing
>>> fetch_conf = {'url': get_path('feed.xml')}
>>> filter_rule = {
...     'field': 'y:published', 'op': 'before', 'value': '2/5/09'}
>>> sub_conf = {'path': 'content.value'}
>>> match = r'(.*href=")([\w:/.@]+)(".*)'
>>> regex_rule = {'field': 'content', 'match': match, 'replace': '$2'}
>>> sort_conf = {'rule': {'sort_key': 'content', 'sort_dir': 'desc'}}
>>>
>>> ### Create a workflow ###
>>> #
>>> # The following workflow will:
>>> #   1. fetch the rss feed
>>> #   2. filter for items published before 2/5/2009
>>> #   3. extract the path `content.value` from each feed item
>>> #   4. replace the extracted text with the last href url contained
>>> #      within it
>>> #   5. reverse sort the items by the replaced url
>>> #
>>> # Note: sorting is not lazy so take caution when using this pipe
>>> stream = fetch.pipe(conf=fetch_conf)
>>> filtered = filter.pipe(stream, conf={'rule': filter_rule})
>>> extracted = (subelement.pipe(i, conf=sub_conf, emit=True) for i in filtered)
>>> flat_extract = chain.from_iterable(extracted)
>>> matched = (regex.pipe(i, conf={'rule': regex_rule}) for i in flat_extract)
>>> flat_match = chain.from_iterable(matched)
>>> sorted_match = sort.pipe(flat_match, conf=sort_conf)
>>> next(sorted_match)
{'content': 'mailto:mail@writetoreply.org'}
```

Notes

---

1 See Design Principles for explanation on *pipe* types and sub-types